

Milestone-3 & 4

Backend and Frontend for ExoHabitAI

Module 5: Backend API Integration (Flask)

1. Objective

The objective of the backend module is to expose a trained machine learning model through a Flask-based REST API. The backend accepts exoplanet planetary and stellar parameters and returns habitability predictions along with confidence scores. Additionally, it supports ranking multiple exoplanets based on their predicted habitability scores.

2. Backend Environment Setup

The backend system is developed using Python and Flask. The following libraries are required:

- Flask – for REST API development
- Flask-CORS – to enable cross-origin resource sharing
- joblib – for loading the trained machine learning model
- pandas – for data handling and preprocessing
- numpy – for numerical computations

Before running the backend server, the Python virtual environment must be activated and all dependencies must be installed correctly.

3. Backend Folder Structure

The backend follows a modular and organized folder structure:

backend/

```
|— app.py
|— utils.py
```

models/

```
|— best_exohabit_model.pkl
```

- app.py contains the Flask application and API endpoints
- utils.py handles input validation and ranking logic
- models/ stores the trained machine learning model

4. Loading the Trained Machine Learning Model

The trained exoplanet habitability prediction model is stored as a .pkl file. The model is loaded during application startup using the joblib library. Loading the model once at startup improves performance and ensures faster prediction responses.

5. Flask Application Initialization

The Flask application is initialized in `app.py`. Cross-Origin Resource Sharing (CORS) is enabled to allow requests from the frontend interface.

A default root endpoint (`/`) is provided to verify that the backend API is running successfully.

6. API Endpoints Design

6.1 `/predict` Endpoint

- **HTTP Method:** POST
- **Input:** JSON object containing exoplanet planetary and stellar parameters
- **Output:**
 - Habitability prediction (Habitable / Not Habitable)
 - Confidence score (probability value)

This endpoint validates the input data, preprocesses features, applies the machine learning model, and returns prediction results.

6.2 `/rank` Endpoint

- **HTTP Method:** POST
- **Input:** JSON array containing multiple exoplanet objects
- **Output:** Ranked list of exoplanets based on habitability scores

This endpoint enables comparison and ranking of multiple exoplanets.

7. Input Validation and Feature Engineering

Input validation is implemented in `utils.py` and includes:

- Checking for missing parameters
- Ensuring numeric fields are properly converted
- Verifying categorical parameters such as star type

Additional derived features such as **habitability score** and **orbital stability** are computed to align with the model's training process. Missing or invalid values are handled gracefully.

8. Response Structure

All backend responses follow a consistent JSON format containing:

- Status message
- Prediction result
- Confidence score

This structured response ensures seamless frontend integration.

9. Backend Testing

The backend API was tested using:

- Browser-based testing for root endpoint
- Frontend integration tests
- JSON-based requests

The backend handles incorrect or incomplete inputs without crashing.

Module 6: Frontend UI Development

1. Objective

The objective of the frontend module is to design a user-friendly web interface that allows users to input exoplanet data and visualize habitability predictions and rankings. The frontend communicates with the backend API to display real-time results.

2. Frontend Folder Structure

The frontend follows a standard web application structure:

frontend/

```
|— index.html  
|— style.css  
|— script.js
```

- index.html defines the user interface
- style.css handles styling and animations
- script.js manages logic and backend communication

3. User Interface Layout Design

The frontend interface is designed using:

- HTML5 for structure
- Bootstrap 5 for responsive layout
- Custom CSS for a space-themed appearance

The UI is divided into:

- Single Planet Prediction
- JSON Input Prediction
- Multi-Planet Ranking

4. Input Form Creation

The input forms allow users to enter:

- Planetary parameters (radius, mass, density, orbital period, etc.)
- Stellar parameters (temperature, luminosity, metallicity, star type)

Client-side validation ensures that all required fields are filled before submitting requests.

5. Frontend and Backend Integration

The frontend communicates with the backend using the **Fetch API**.

- The `/predict` endpoint is used for single-planet predictions
- The `/rank` endpoint is used for ranking multiple exoplanets

All API calls are handled asynchronously without page reloads.

6. Display of Prediction Results

Prediction results are displayed using styled output cards showing:

- Habitability status
- Confidence score

Graphical visualizations include:

- Radar charts for parameter contribution analysis
- Bar charts for habitability score comparison

Chart.js is used for all visual representations.

7. Error Handling

User-friendly error messages are displayed for:

- Invalid inputs
- Incorrect JSON format
- Backend communication errors

This ensures a smooth and robust user experience.

8. Styling and Responsiveness

The frontend uses:

- Glassmorphism cards
- Animated star background
- Responsive design for different screen sizes

The interface maintains a professional, futuristic, and scientific aesthetic.

9. Integration Testing

Complete end-to-end testing was performed for the following workflow:

User Input → Frontend → Backend → Machine Learning Model → Prediction → UI Output

All components work together correctly with accurate and consistent results.

Conclusion

ExoHabitAI successfully integrates machine learning with full-stack web development to predict and rank exoplanet habitability. The backend ensures reliable and efficient model inference, while the frontend provides an intuitive and visually appealing interface. This project demonstrates practical implementation of machine learning deployment, API development, and interactive data visualization.